Chris Cameron

chrisjamescameron1@gmail.com · github.com/ccam80

AI-native researcher, developer, and mechatronics engineer completing a PhD in biomedical engineering (completion early 2027), and creator of Cubie. Part-time lecturer in mechatronics engineering, including computer architecture, C programming, circuit design, and embedded systems.

Cubie: numerical integration on GPUs

github.com/cubiepy/cubie

I built Cubie, an open-source Python ODE solver for consumer NVIDIA GPUs, to support electrophysiology simulations for a real-time application. Current benchmarks (draft, unpublished) show **a 10,000× speedup** over CPU implementations (SciPy, MATLAB) and **100×** over the nearest GPU competitor, DiffEqGPU.jl, on real-world problems.

Creating CUDA-shaped algorithms, coalescing memory accesses, optimising memory placements and extracting constants was not enough to get to domain-leading performance. Further speedups were gained by:

- using NVIDIA profiling tools to trace bottlenecks and stalls to individual instructions;
- rolling or unrolling loops by weighing loop-body instruction counts against instruction-cache size;
- *decreasing* occupancy to reduce L2 cache misses;
- ordering and fusing operations at a whole-kernel level to shorten live ranges and reduce register spilling; and
- placing buffers heuristically to trade off guaranteed-L1 shared memory against possible-L2/VRAM local memory, based on working-set size.

Cubie goes fast, but it is a scientific tool. The codebase is complex, and correctness is critical. To achieve and maintain correctness, I had to:

- test numerical results against known references and custom CPU harnesses;
- review every pull request line by line (AI agents are prolific but often incorrect); and
- build approval processes including a ratcheting benchmark per PR and automated correctness checks.

Open-source contributions

numba-cuda / numba-cuda-mlir

Many merged and open pull requests address compiler performance, typing and lowering correctness, AST transforms, and kernel performance controls.

SymPy and Lcapy — minor edits.